IU2541230787 - CPP74 - SUBMISSION TEMPLATE

Project Name: Employee Leave Management

Enrollment No.: IU2541230787

Student Name: Krish Singh

Date of Submission: 15 October 2026

1. Project Summary

Problem Statement: Manual leave handling causes miscounted days, double bookings, overdrawn balances and no audit trail. A console program is needed to apply, approve and track leave with automatic rule checking and permanent storage.

Objectives: Apply OOP (inheritance, polymorphism, dynamic memory, file handling, exceptions) to build a reliable leave system with the required 9-option menu, UML documentation and a tested implementation.

Key Features:

- Employees, managers, holidays, leave requests and approvals with full validation.
- Five leave types as polymorphic policies (quota, notice, back-dating, max days).
- Working-day counting that skips weekends and holidays; overlap and balance checks.
- Manager authorisation by department and a department cover rule.
- Reports: balance, department summary, who is on leave, monthly calendar, summary, CSV export, audit log.
- Atomic file saving with checksum, corrupt-line recovery, backup and demo data.
- 41 end-to-end tests, 84 unit checks with ASan/UBSan, 9 compile-fail proofs.

2. Class Summary

Class	Purpose / Responsibility	Key Data Members	Key Member Functions
Record / Person	Abstract roots: identity, validation, virtual display/serialize	id_, name_, department_	display(), serialize(), role(), matches()
Employee	Staff member who applies for leave	designation_, email_, joined_	Employee(), setEmail(), display()
Manager	Decides leave; HR may decide for all departments	title_, hr_	authorises(), role()
LeaveRequest	One leave application and its status	employeeId_, type_, from_, to_, days_, status_	calculateDays(), overlaps(), setStatus()
Approval	Manager's decision record	requestId_, managerId_, decision_, comment_	approve(), reject()
LeaveSystem	Controller: rules, data ownership, reports	5 PtrList members, policies_, fm_, audit_	applyLeave(), decide(), cancelLeave(), balance(), report functions, save(), load()
FileManager	Atomic save/load with checksum	dir_	save<T>(), readLines(), backup()
LeavePolicy (+5)	Leave-type rules via virtual functions	-	yearlyQuota(), maxConsecutive(), minNoticeDays(), check()
PtrList<T>, Holiday, ConsoleUI,	Dynamic array; public holiday;	data_/size_/cap_; date_/name_;	add(), removeAt(); run(); log();

AuditLog, Date	menus; change log; calendar	sys_; path_; y_/m_/d_	addDays()

3. Project Flow

USER -> ConsoleUI (main menu) -> LeaveSystem (rules) -> Employee / Manager / LeaveRequest / Approval / Holiday objects -> FileManager -> FILE (Data/*.dat)

4. UML Deliverables

Diagram	File Name	Completed
Use Case Diagram	UML/Use_Case_Diagram.png	☑️
Class Diagram	UML/Class_Diagram.png	☑️
Sequence Diagram 1	UML/Sequence_Diagram_1.png	☑️
Sequence Diagram 2	UML/Sequence_Diagram_2.png	☑️

5. C++ Concepts Demonstrated

Concept	Where Used in My Project	Explanation
Encapsulation	All classes; Person::name_, Record::id_	Data is private; only validating setters change it. No setter exists for id.
Constructor	Employee, Manager, LeaveRequest, Approval, Holiday, Date	Default and parameterized constructors; constructors validate input and throw on bad data.
Destructor	Record, LeavePolicy (virtual); PtrList, LeaveSystem	Virtual destructors make delete through base pointers safe; PtrList/LeaveSystem free their dynamic memory.
Inheritance	Record -> Person -> Employee/Manager; Record -> LeaveRequest/Approval/Holiday; LeavePolicy -> 5 types	Common behaviour in base classes, specialised in derived classes.
Runtime Polymorphism	Display > Everything (Record*); LeavePolicy::check()	One loop over Record* prints five different kinds; each leave type returns its own limits.
Dynamic Memory	PtrList<T>::grow(), LeaveSystem::policies_	new T*[2*cap] and delete[]; objects created with new and deleted by the owning list; ASan reports no leaks.
File Handling	FileManager, LeaveSystem::save()/load()	Tab-separated .dat files with header+checksum, atomic rename, load at start, corrupt lines skipped.
Exception Handling	errors.h, ConsoleUI::guarded()	ValidationError, RuleError, NotFoundError, FileError, ParseError caught at the menu; data stays consistent.

6. Testing

Test ID	Operation	Input	Expected Output	Actual Output	Pass/Fail
T01	Add employee (valid)	Krish Singh / Engineering / Developer / krish@acme.com / 2025-08-01	Added employee E001	As expected	Pass
T02	Add employee: invalid e-mail	e-mail 'bad' then valid	Enter a valid e-mail ; Added employee E001	As expected	Pass
T03	Display all employees	Display > Employees	Aarav Sharma ; Neha Gupta ; 8 record(s)	As expected	Pass
T04	Search employee by department	text 'sales'	Vikram Rao ; Sana Khan ; Arjun Verma ; 3 record(s)	As expected	Pass
T05	Update employee	E001: new designation 'Senior Engineer', others	Updated E001 ; Senior Engineer	As expected	Pass

		kept			
T06	Update pending leave request	R0005 -> 2026-11-09..2026-11-10	Updated R0005 - now 2 working day(s)	As expected	Pass
T07	Delete employee with pending leave	E003 has pending/approved leave	still has	As expected	Pass
T08	Apply for leave (valid)	E001 AL 2026-11-16..2026-11-17 'Visit home'	submitted: 2 working day(s) ; Pending	As expected	Pass
T09	Apply for leave: overlap	E001 AL 2026-10-30 (already on R0001)	overlap with Approved request R0001	As expected	Pass
T10	Apply: insufficient balance	11 days, then 5 more with only 4 left	submitted: 11 working day(s) ; insufficient Annual Leave balance	As expected	Pass
T11	Manager approves request	R0003 by M001	R0003 Approved	As expected	Pass
T12	Manager rejects (reason mandatory)	R0006 by M001; empty reason first	cannot be empty ; R0006 Rejected	As expected	Pass
T13	Department cover rule	3rd Engineering approval on same day blocked	R0011 Approved ; cover rule	As expected	Pass
T14	Cancel a pending request	R0005	Cancelled. Balance restored ; Cancelled	As expected	Pass
T15	Persistence across restarts	run 1: add + save, quit; run 2: display	Krish Singh ; 1 record(s)	As expected	Pass

7. Deployment

Compiler / IDE Used: g++ 13.3 (C++17), any text editor / VS Code

Compilation Command: g++ -std=c++17 -Iinclude -o elm src/*.cpp (or: make)

Executable Name: elm (elm.exe on Windows via BUILD_WINDOWS.bat)

Data File(s): Data/employees.dat, managers.dat, requests.dat, approvals.dat, holidays.dat, audit.log

8. Student Declaration

I understand the submitted project and can explain its classes, data members, member functions, file handling and UML diagrams.

Student Signature: _____